


MCP TUTORIAL

# Build & Deploy Your Own MCP Server

Expose your RAG pipeline as a tool - connect it to any AI agent

**Swipe to build it from scratch →**

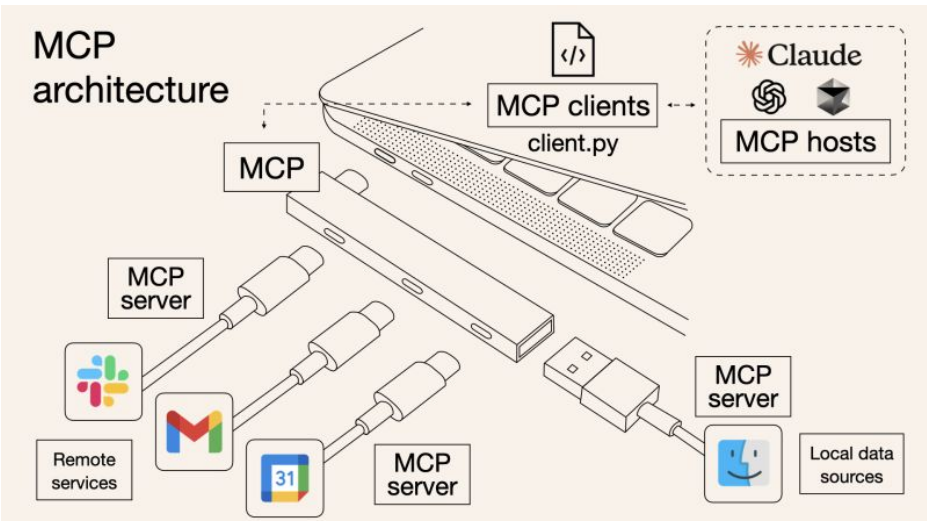
AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

CONCEPT

# What is MCP?

MCP (Model Context Protocol) is an open standard that lets AI models talk to external tools and data sources through a unified interface.

**Think of it like USB-C - one standard plug that works everywhere.**




## 3 CORE PIECES

- **MCP Server** → exposes your tools
- **MCP Client** → the AI agent (Claude, etc.)
- **Transport** → how they talk (stdio, SSE, HTTP)



AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

CAPABILITIES

# What Can MCP Expose?

MCP servers can expose three types of things to an AI agent:

```
MCP Server
├── Tools       → functions the LLM can call
│   └── search_documents(query)
│   └── ask_rag(question)
│
├── Resources   → data the LLM can read
│   └── file://docs/readme.md
│   └── db://customers/table
│
└── Prompts     → reusable prompt templates
    └── summarize_document(doc_id)
```

◎ **For this carousel we focus on Tools - specifically wrapping our RAG pipeline as a callable tool.**




AI Coach John Follow

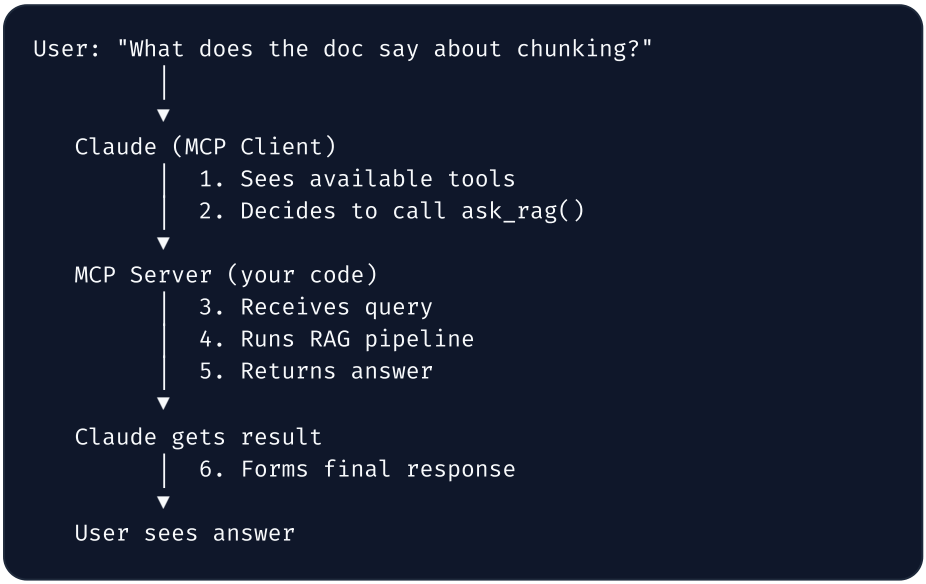
PROITBRIDGE Strive For Better Future

Repost

ARCHITECTURE

# How MCP Works (Flow)

Here's exactly what happens when Claude uses your MCP tool:




**Your job: build steps 3-5. MCP handles everything else.**




AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

NETWORKING

# Transport Types

Transport = how client and server communicate. Pick based on where you deploy.

```
stdio (local)
─────────────

Claude Desktop ──► stdin/stdout ──► your script
Best for: local tools, development

SSE (network)
─────────────

Claude / Agent ──► HTTP + SSE ──► your server
Best for: remote servers, APIs

Streamable HTTP (modern)
────────────────────────

Agent ──► HTTP POST ──► your server
Best for: production, cloud deployments
```

**We'll implement all three. Start with stdio, then upgrade.**




AI Coach John Follow

PROITBRIDGE
Strive For Better Future

Repost

STEP 1

# Install FastMCP

FastMCP is the easiest Python library to build MCP servers. One decorator and your function becomes a tool.

```
pip install fastmcp
pip install langchain langchain_community
pip install langchain_huggingface faiss-cpu
pip install langchain_groq
```

✓ **Check it works:**

```
fastmcp version
```

**PROJECT STRUCTURE WE'LL BUILD:**

```
rag_mcp/
├── rag_pipeline.py    ← your RAG logic
├── server.py          ← MCP server
├── Dockerfile         ← for cloud deploy
└── claude_config.json
```




AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

</> STEP 2

# Build the RAG Pipeline Module

First build your RAG pipeline as a standalone module. This is what the MCP tool will call internally.

```
# rag_pipeline.py
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_groq import ChatGroq
from langchain.chains import RetrievalQA

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2"
)
vectorstore = FAISS.load_local(
    "faiss_index", embeddings,
    allow_dangerous_deserialization=True
)
llm = ChatGroq(model_name="gemma2-9b-it", temperature=0)

rag = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    return_source_documents=True
)

def query_rag(question: str) → str:
    result = rag.invoke({"query": question})
    return result["result"]
```



AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

STEP 3

# Create Your MCP Server

Now wrap the RAG pipeline as an MCP tool using FastMCP. One `@mcp.tool()` decorator is all it takes.

```
# server.py
from fastmcp import FastMCP
from rag_pipeline import query_rag

mcp = FastMCP(
    name="RAG Assistant",
    instructions="Use ask_rag to answer questions"
)

@mcp.tool()
def ask_rag(question: str) → str:
    """
    Search the document knowledge base
    and answer a question.
    """
    return query_rag(question)

if __name__ == "__main__":
    mcp.run()   # defaults to stdio transport
```

**That's your first MCP server. Three lines of real logic.**




AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

STEP 4

# Add More Tools

One server can expose multiple tools. Add a search-only tool and a metadata tool alongside ask_rag.

```
@mcp.tool()
def search_docs(query: str, top_k: int = 3) → list[str]:
    """Return raw document chunks relevant to the query."""
    docs = vectorstore.similarity_search(query, k=top_k)
    return [d.page_content for d in docs]

@mcp.tool()
def list_sources() → list[str]:
    """Return all document sources loaded in the knowledge
base."""
    # pull unique sources from metadata
    docs = vectorstore.docstore._dict.values()
    sources = list({d.metadata.get("source","unknown")
                    for d in docs})
    return sources
```

**Now Claude can choose between: ask a question, get raw chunks, or list sources - depending on what fits the task.**


AI Coach John Follow
PROITBRIDGE Strive For Better Future
Repost

STEP 5

# Run Locally via stdio

stdio is the simplest transport. The MCP client launches your script as a subprocess and talks via stdin/stdout.

```
# Test it directly from terminal
python server.py

# Or use FastMCP's built-in dev inspector:
fastmcp dev server.py
```

>_ **Output:**

```
✅ RAG Assistant MCP server running
📋 Tools available:
   • ask_rag(question)
   • search_docs(query, top_k)
   • list_sources()
🔌 Transport: stdio
```

**The inspector opens a browser UI where you can call each tool manually - great for debugging.**




AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

STEP 6

# Connect to Claude Desktop

Claude Desktop reads a config file to know which MCP servers to launch. Add your server there.

```
// ~/Library/Application Support/Claude/claude_desktop_config.json
// (Mac) or %APPDATA%\Claude\claude_desktop_config.json (Windows)

{
  "mcpServers": {
    "rag-assistant": {
      "command": "python",
      "args": ["/full/path/to/rag_mcp/server.py"],
      "env": {
        "GROQ_API_KEY": "your_groq_key"
      }
    }
  }
}
```

**Restart Claude Desktop → you'll see a icon → your tools appear. Now Claude can call `ask_rag` directly in conversation.**




AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

STEP 7

# Add SSE Transport (Network)

SSE lets your server run as a web service so any HTTP client - not just Claude Desktop - can connect to it.

```
# server.py - add SSE support
from fastmcp import FastMCP

mcp = FastMCP("RAG Assistant")
# ... same tools as before ...

if __name__ == "__main__":
    import sys
    if "--sse" in sys.argv:
        mcp.run(transport="sse", host="0.0.0.0", port=8000)
    else:
        mcp.run()  # stdio for local
```

```
# Run it:
python server.py --sse

# Server live at:
# http://localhost:8000/sse
```

**Now any agent that speaks MCP over HTTP can connect to your RAG pipeline.**




AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

STEP 8

# Input Validation & Error Handling

Always validate inputs and return clean errors. MCP surfaces these back to the agent so it can retry or explain to the user.

```
from fastmcp import FastMCP
from fastmcp.exceptions import ToolError

@mcp.tool()
def ask_rag(question: str) → str:
    """Answer a question from the knowledge base."""

    if not question or len(question.strip()) < 3:
        raise ToolError("Question too short.")

    if len(question) > 500:
        raise ToolError("Question too long.")

    try:
        return query_rag(question)
    except Exception as e:
        raise ToolError(f"RAG pipeline error: {str(e)}")
```

**Claude sees the `ToolError` message and can tell the user exactly what went wrong.**

STEP 9

# Dockerize the Server

Package the server into a Docker container for consistent, portable deployment anywhere - local, VPS, or cloud.

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY . .

RUN pip install --no-cache-dir fastmcp langchain \
    langchain_community langchain_huggingface \
    faiss-cpu langchain_groq

EXPOSE 8000
CMD ["python", "server.py", "--sse"]
```

```
# Build and run:
docker build -t rag-mcp .

docker run -p 8000:8000 \
  -e GROQ_API_KEY=your_key \
  -v $(pwd)/faiss_index:/app/faiss_index rag-mcp
```

Server is running at `http://localhost:8000/sse`


AI Coach John Follow
PROITBRIDGE Strive For Better Future
Repost

STEP 10

# Deploy to Cloud (VPS/Railway)

Once Dockerized, deploy to any cloud provider. Here's the pattern using Railway (free tier available).

```
# Option A - Railway CLI
railway login
railway init
railway up

# Option B - Any VPS (DigitalOcean, EC2, etc.)
scp -r rag_mcp/ user@your-server:/app
ssh user@your-server
cd /app && docker compose up -d
```

```
# docker-compose.yml for VPS:
services:
  rag-mcp:
    build: .
    ports:
      - "8000:8000"
    environment:
      - GROQ_API_KEY=${GROQ_API_KEY}
    restart: always
    volumes:
      - ./faiss_index:/app/faiss_index
```

STEP 11

# Connect Remote Server to Claude

Point Claude Desktop at your cloud-hosted SSE server instead of a local script.

```
// claude_desktop_config.json
{
  "mcpServers": {
    "rag-assistant-remote": {
      "transport": {
        "type": "sse",
        "url": "https://your-domain.com/sse"
      }
    }
  }
}
```

```
// For protected servers add auth header:
{
  "transport": {
    "type": "sse",
    "url": "https://your-domain.com/sse",
    "headers": {
      "Authorization": "Bearer your_secret_token"
    }
  }
}
```

**Restart Claude Desktop - same experience, tools now running in the cloud**

STEP 12

# Add a Resource (Bonus)

Beyond tools, expose your document index as a readable resource - agents can browse available docs before deciding what to search.

```
from fastmcp import FastMCP
from fastmcp.resources import Resource

@mcp.resource("docs://index")
def get_doc_index() → str:
    """List all documents in the knowledge base."""
    docs = vectorstore.docstore._dict.values()
    lines = []
    for d in docs:
        src = d.metadata.get("source", "unknown")
        pg  = d.metadata.get("page", "-")
        lines.append(f"• {src} (page {pg})")
    return "\n".join(set(lines))
```

> *"Let me check what documents are available... then I'll search the right one."*

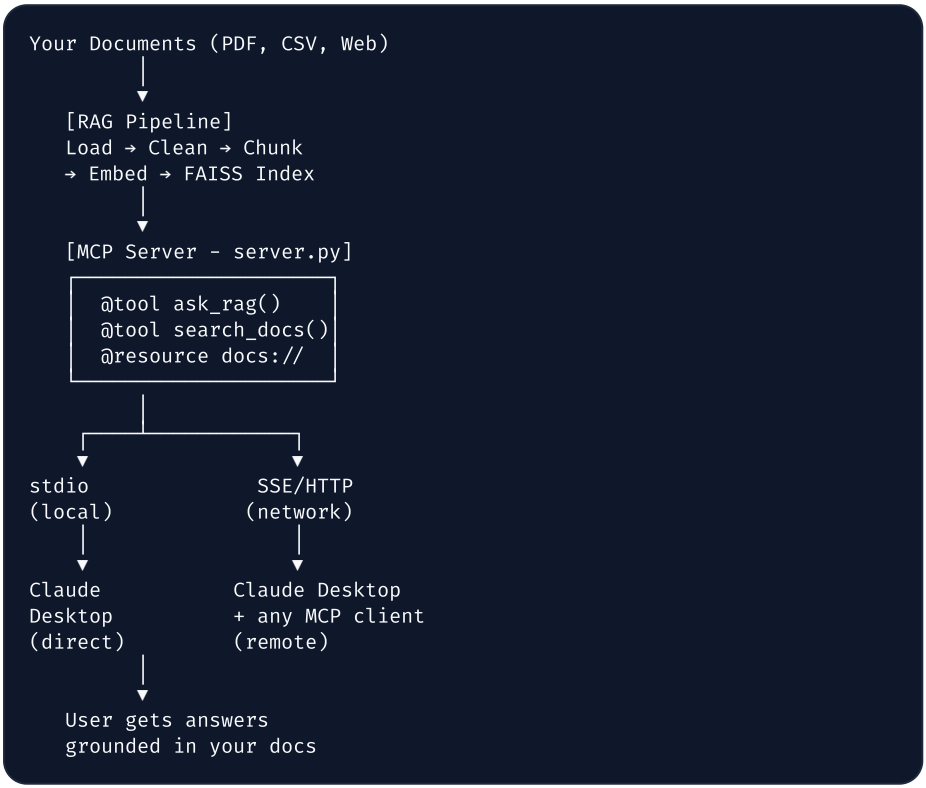


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

BIG PICTURE

# Full Architecture Diagram

Here's how all the pieces fit together end to end:







AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

TROUBLESHOOTING

# Debugging Checklist

When things break, check these in order:

### TOOLS NOT SHOWING IN CLAUDE?

- → Check config file path is absolute
- → Restart Claude Desktop fully
- → Run: `fastmcp dev server.py`

### RAG RETURNS WRONG ANSWERS?

- → Check faiss_index path is correct
- → Increase k ( `search_kwargs={"k": 8}` )
- → Check chunk size in your pipeline

### SSE SERVER NOT REACHABLE?

- → Check port 8000 is open in firewall
- → Confirm docker container is running
- → Check logs: `docker logs rag-mcp`




AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

CONCLUSION

# Summary & What's Next

You now have a complete MCP server that exposes your RAG pipeline as tools - locally and in the cloud.

## WHAT YOU BUILT:

- ✓ RAG pipeline module
- ✓ MCP server (3 tools)
- ✓ stdio transport
- ✓ SSE transport
- ✓ Docker container
- ✓ Cloud deployment
- ✓ Error handling

## WHAT TO ADD NEXT:

- → API key auth on SSE
- → Logging & tracking
- → Auto re-index docs
- → Conversation memory
- → Claude.ai connector

**Follow for more AI engineering content →**




AI Coach John Follow

PROITBRIDGE
Strive For Better Future

Repost